Chương 8: Template Method Pattern



Template Method Pattern – Đóng gói thuật toán

Chúng ta đang trên một chuỗi các đóng gói: chúng tôi đã đóng gói việc tạo đối tượng (Factory method (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html)), đóng gói lời gọi phương thức (Command (https://toihocdesignpattern.com/chuong-6-command-pattern-dong-goi-yeu-cau.html)), đóng gói giao diện phức tạo (Facade pattern (https://toihocdesignpattern.com/chuong-7-adapter-va-facade-pattern-tro-nen-thich-nghi-phan-2.html)), vịt, pizza … tiếp theo sẽ là gì? Với **Template method pattern**, chúng tôi sẽ tiếp tục đóng gói các phần của thuật toán để các lớp con có thể tự móc (hook) vào một tính toán bất cứ lúc nào chúng muốn. Chúng tôi thậm chí sẽ tìm hiểu về một nguyên tắc thiết kế lấy cảm hứng từ Hollywood.

Đây là thời gian cần nhiều Caffeine

Một số người không thể sống thiếu cà phê; một số người khác không thể sống mà không có trà. Thành phần phổ biến của chúng là gì? Tất nhiên Caffeine!

Nhưng còn nhiều hơn thế; trà và cà phê được làm theo những cách rất giống nhau. Hãy cùng kiểm tra nào:

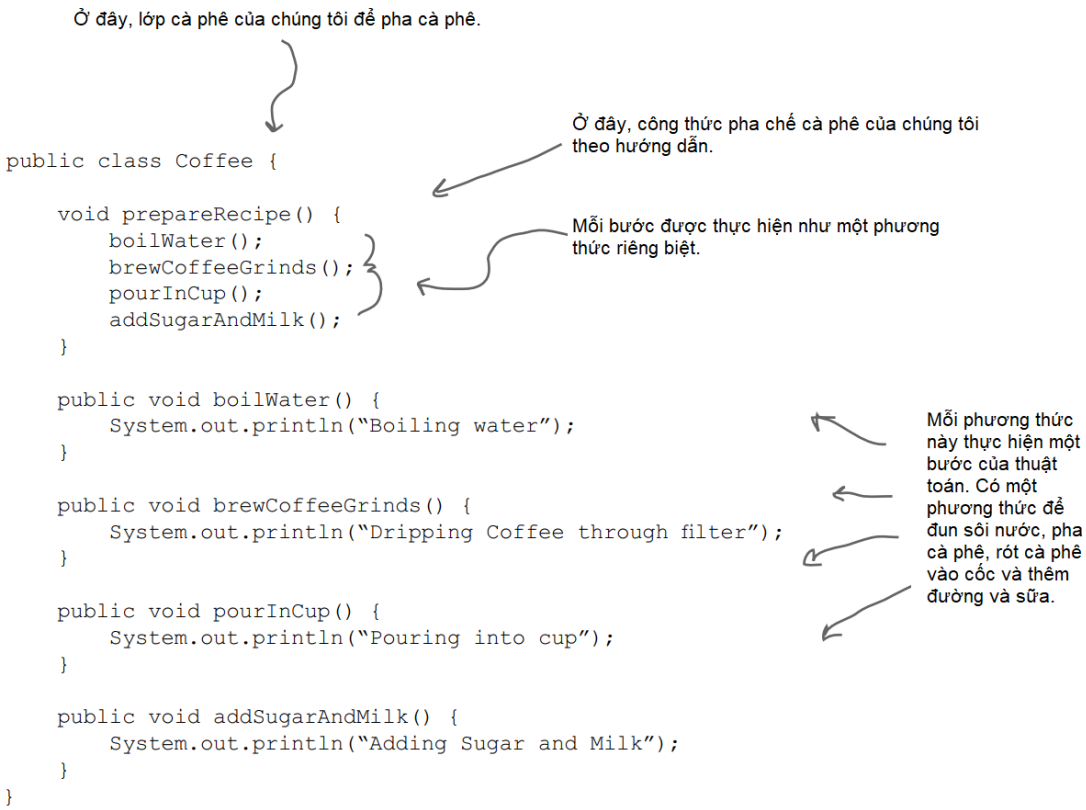
Hướng dẫn pha chế Starbuzz Coffee Barista

> ***Baristas! Hãy làm theo các công thức này chính xác khi chuẩn bị đồ uống Starbuzz.***
>
> ***Starbuzz Coffee Recipe***
>
> *(1) Đun sôi nước*
>
> *(2) Pha cà phê trong nước sôi*
>
> *(3) Rót cà phê vào cốc*
>
> *(4) Thêm đường và sữa*
>
> ***Starbuzz Tea Recipe***
>
> *(1) Đun sôi một ít nước*
>
> *(2) Bỏ trà vào nước sôi*
>
> *(3) Rót trà vào cốc*
>
> *(4) Thêm chanh*

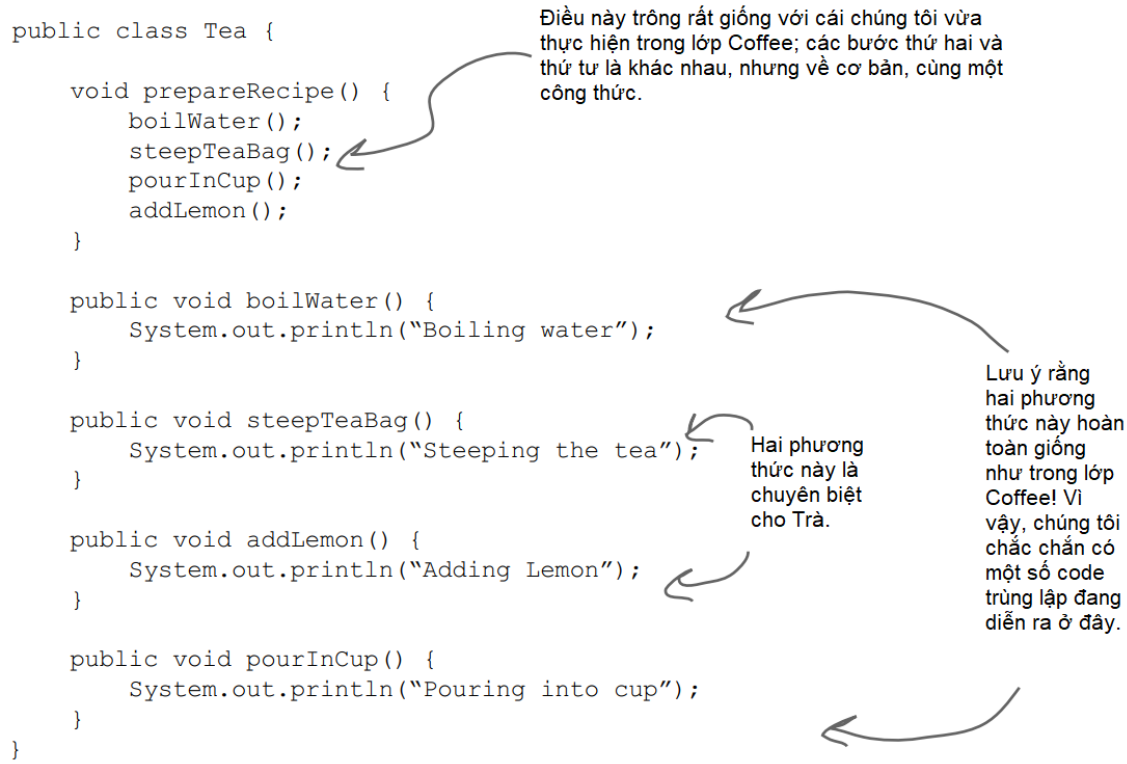
Viết một số lớp cà phê và trà (bằng Java)

Hãy viết một số dòng code để tạo ra cà phê và trà.

Ở ĐÂY, LỚP CÀ PHÊ:



VÀ BÂY GIỜ LÀ LỚP TRÀ...

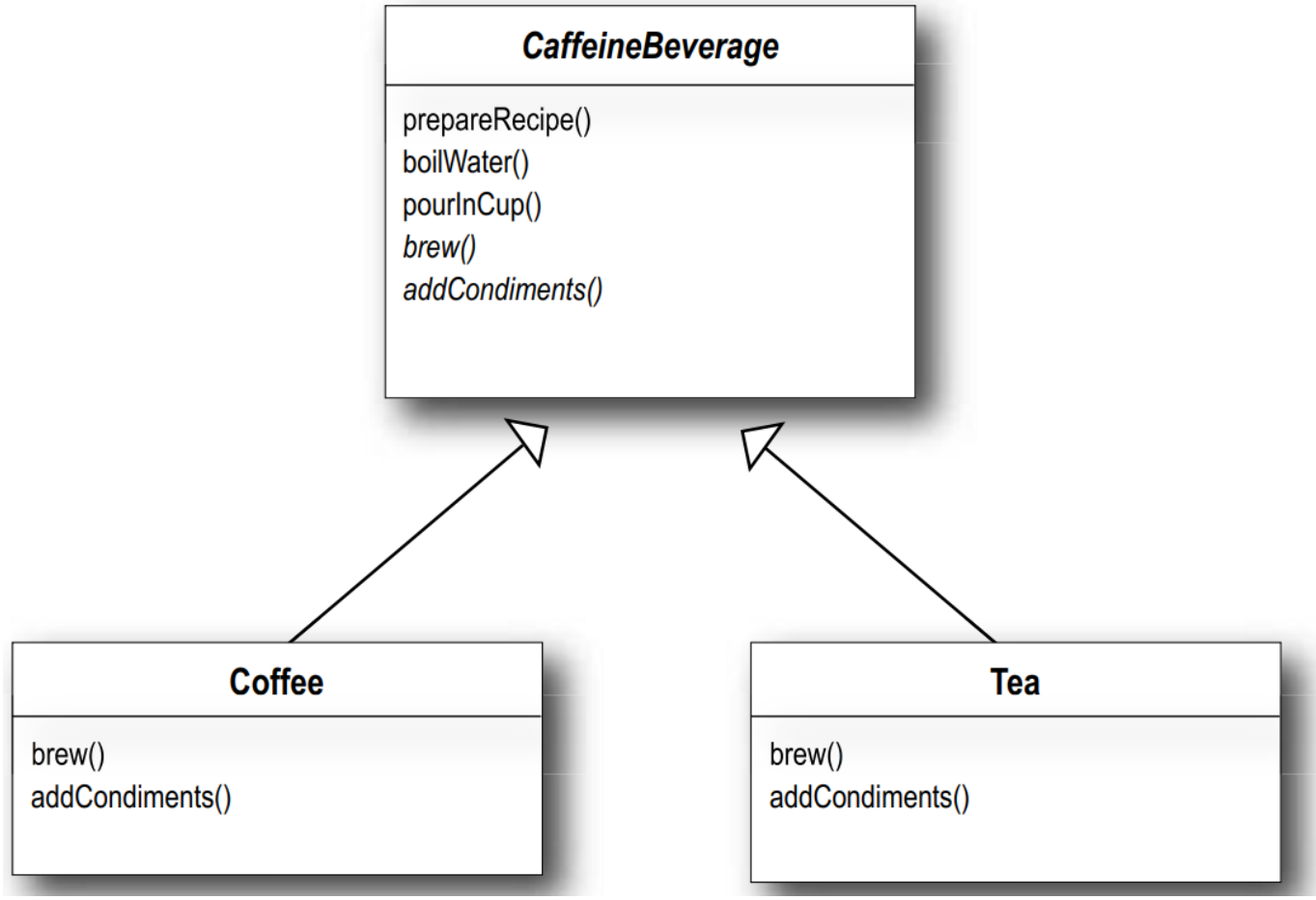


Khi chúng ta thấy việc trùng lặp code, đó là một dấu hiệu tốt để bắt đầu dọn dẹp lại. Có vẻ như ở đây chúng ta nên trừu tượng hóa những phần chung thành một lớp cơ sở vì cà phê và trà trông rất giống nhau?

Bài tập:

Bạn đã thấy rằng các lớp Cà phê và Trà có một chút trùng lặp code. Hãy xem xét lại và vẽ sơ đồ lớp cho thấy cách bạn thiết kế lại các lớp để loại bỏ sự dư thừa.

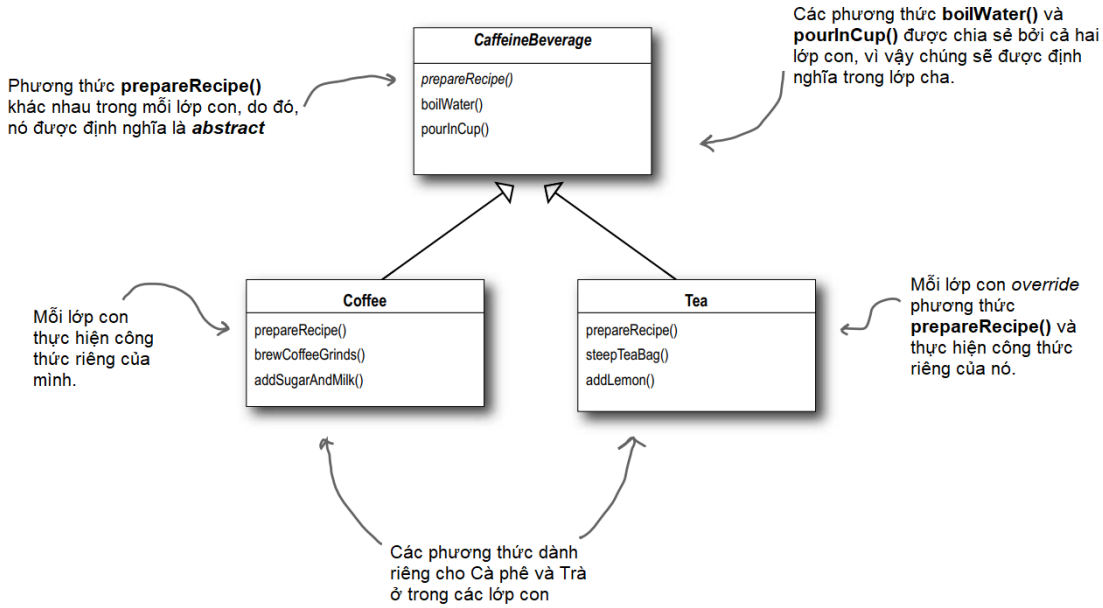
Đáp án:



Thưa ông, tôi có thể trừu tượng Cà phê, Trà của ông không?

Có vẻ như chúng tôi đã có một bài tập thiết kế khá đơn giản trên tay với các lớp Coffee và Tea.

Sơ đồ đầu tiên của bạn có thể trông giống như thế này:

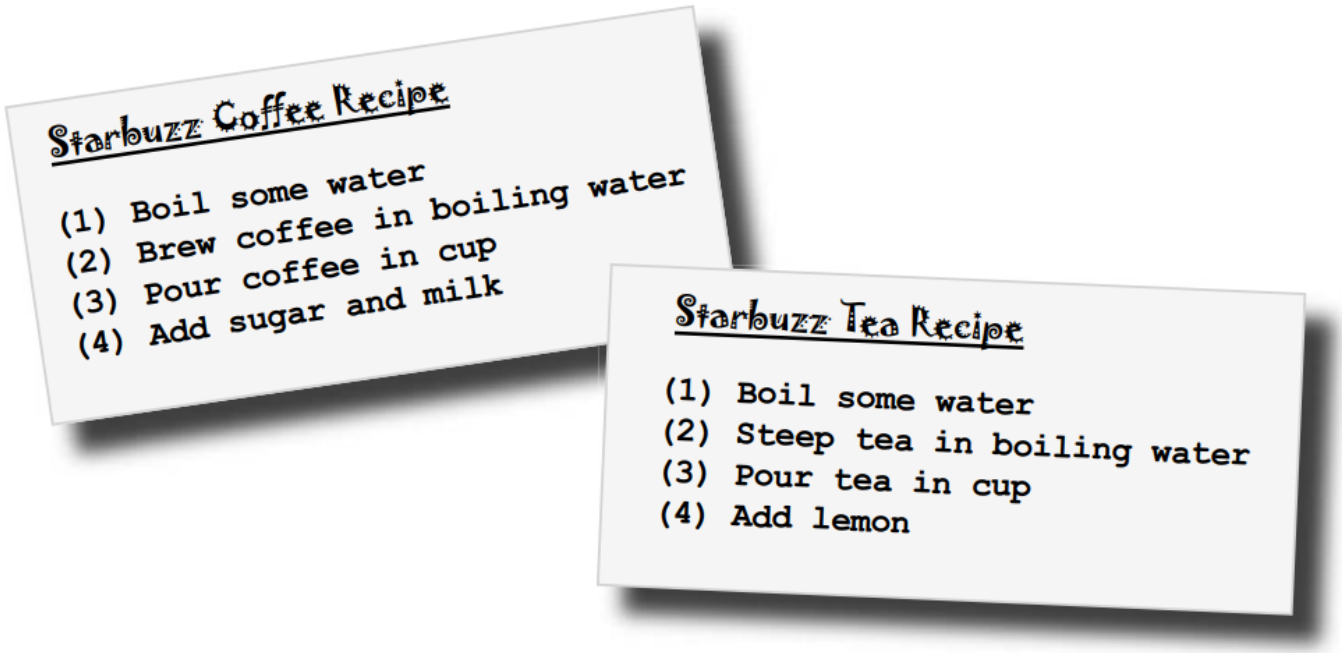


Sử dụng sức mạnh bộ não

Thiết kế lại có tốt không? Hmmmm, hãy xem xét lại. Có phải chúng ta đang xem xét một số điểm chung khác của Cà phê và Trà không? Những điểm chung đó là gì?

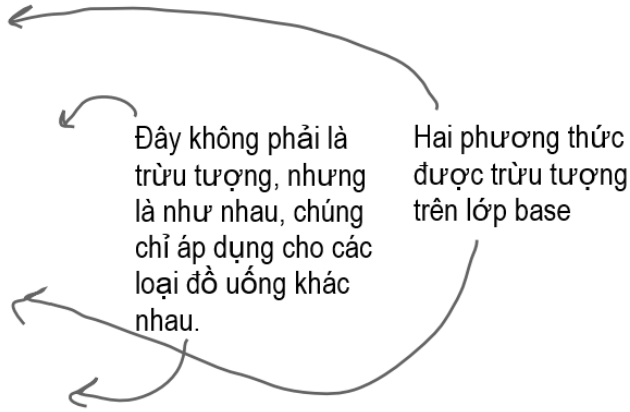
Đưa thiết kế đi xa hơn...

Vậy, những gì làm cà phê và trà có điểm chung? Hãy bắt đầu với các công thức pha chế.



Lưu ý rằng cả hai công thức pha chế đều theo cùng một thuật toán:

1. Đun sôi nước.
2. Sử dụng nước nóng để pha cà phê hoặc trà.
3. Đổ kết quả pha chế vào cốc.
4. Thêm gia vị thích hợp vào đồ uống.



Vì vậy, chúng ta có thể tìm một cách để trừu tượng prepareRecipe() không? Có, hãy tìm hiểu…

Trừu tượng prepareRecipe()

Hãy nghĩ qua việc trừu tượng prepareRecipe() từ mỗi lớp con (nghĩa là các lớp Cà phê và Trà)…

1. Vấn đề đầu tiên chúng tôi gặp phải là Cà phê sử dụng các phương thức brewCoffeeGrinds() và addSugarAndMilk() trong khi Trà sử dụng các phương thức steepTeaBag() và addLemon().

Coffee

```
void prepareRecipe() {
    boilWater();
    brewCoffeeGrinds();
    pourInCup();
    addSugarAndMilk();
}
```

Tea

```
void prepareRecipe() {
    boilWater();
    steepTeaBag();
    pourInCup();
    addLemon();
}
```

Hãy suy nghĩ về điều này: **steeping** và **brewing** không quá khác nhau; hay nói cách khác, chúng giống nhau. Vì vậy, hãy tạo một tên phương thức mới, gọi là **brew()** và chúng tôi sẽ sử dụng cùng phương thức này cho dù chúng tôi pha cà phê hay pha trà.

Tương tự như vậy, thêm "đường và sữa" cũng giống như thêm một "quả chanh": cả hai đều thêm hương vị vào đồ uống. Cũng hãy tạo một tên phương thức mới, **addCondiment()** để xử lý việc này. Vì vậy, phương thức **prepareRecipe()** mới của chúng ta sẽ trông như thế này:

```
void prepareRecipe() {
    boilWater();
    brew();
    pourInCup();
    addCondiments();
}
```

2. Bây giờ chúng ta có một phương thức prepareRecipe() mới, nhưng chúng ta cần điều chỉnh nó trong code. Để làm điều này, chúng ta sẽ bắt đầu với superclass CaffeineBeverage:

CaffeineBeverage is abstract, just like in the class design.

```
public abstract class CaffeineBeverage {

    final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    abstract void brew();

    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }

    void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

Bây giờ, cùng một phương thức prepareRecipe() sẽ được sử dụng để pha cả Trà và Cà phê. PrepareRecipe() được mô tả final vì chúng tôi không muốn các lớp con của chúng tôi có thể override phương thức này và thay đổi công thức! Chúng tôi đã tổng quát hóa các bước 2 và 4 để brew() đồ uống và addCondiment().

Vì Cà phê và Trà xử lý các phương pháp này theo nhiều cách khác nhau, nên chúng sẽ phải được khai báo là abstract. Hãy để các lớp con lo lắng về những thứ đó!

Hãy nhớ rằng, chúng tôi đã chuyển chúng vào lớp CaffeineBeverage (quay lại sơ đồ lớp của chúng tôi).

3. Cuối cùng chúng ta cần phải xử lý với các lớp Cà phê và Trà. Bây giờ chúng dựa vào CaffeineBeverage để cài đặt công thức, vì vậy chúng chỉ cần xử lý brew() và addCondiments():

```
public class Tea extends CaffeineBeverage {
    public void brew() {
        System.out.println("Steeping the tea");
    }
    public void addCondiments() {
        System.out.println("Adding Lemon");
    }
}
```

Như trong thiết kế của chúng tôi, Trà và Cà phê hiện extend lớp CaffeineBeverage.

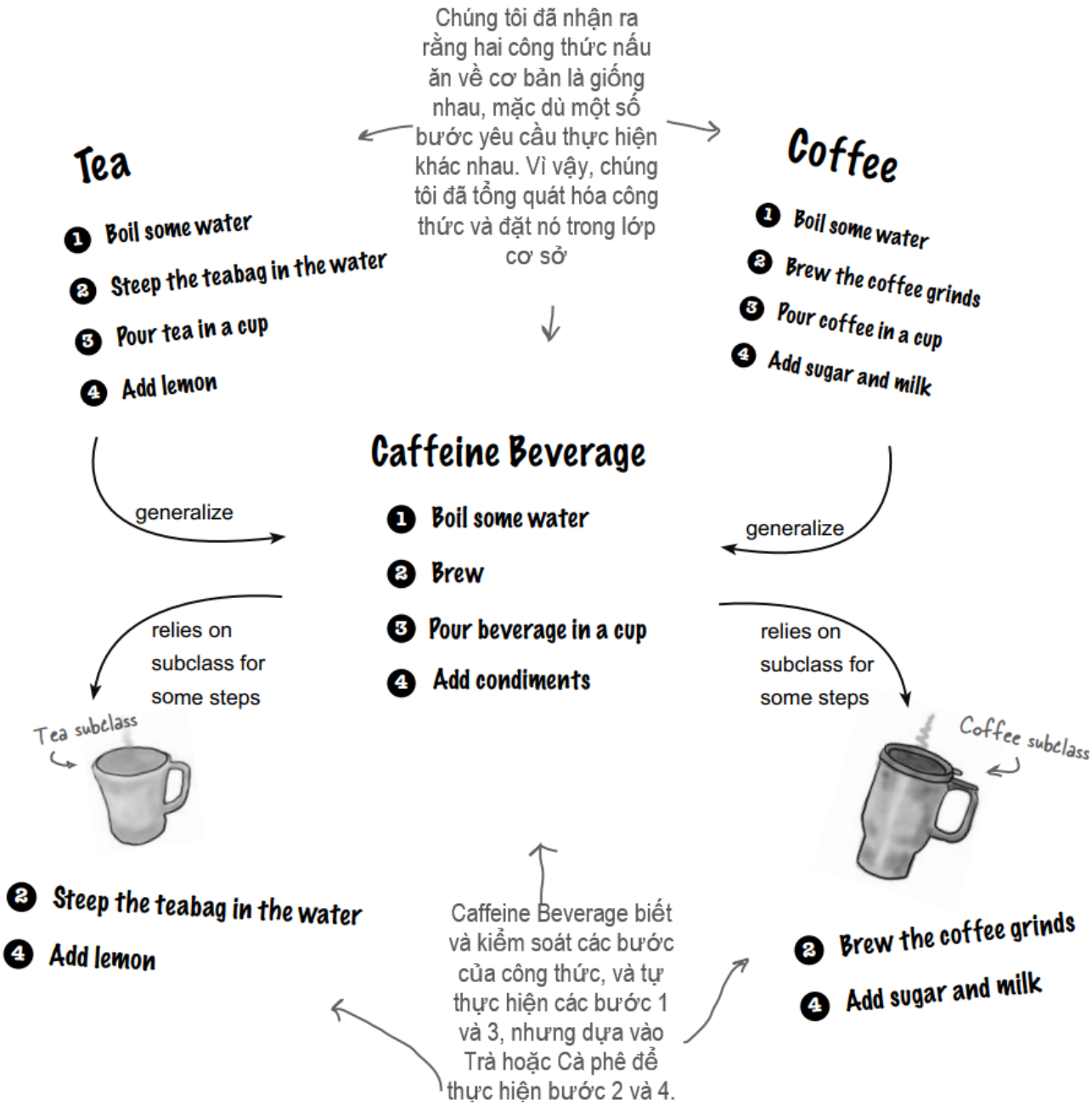
Trà cần định nghĩa brew() và addCondiment() - hai abstract method từ Beverage.

```
public class Coffee extends CaffeineBeverage {
    public void brew() {
        System.out.println("Dripping Coffee through filter");
    }
    public void addCondiments() {
        System.out.println("Adding Sugar and Milk");
    }
}
```

Tương tự đối với Cà phê, ngoại trừ Cà phê gồm cà phê, đường và sữa thay vì túi trà và chanh.

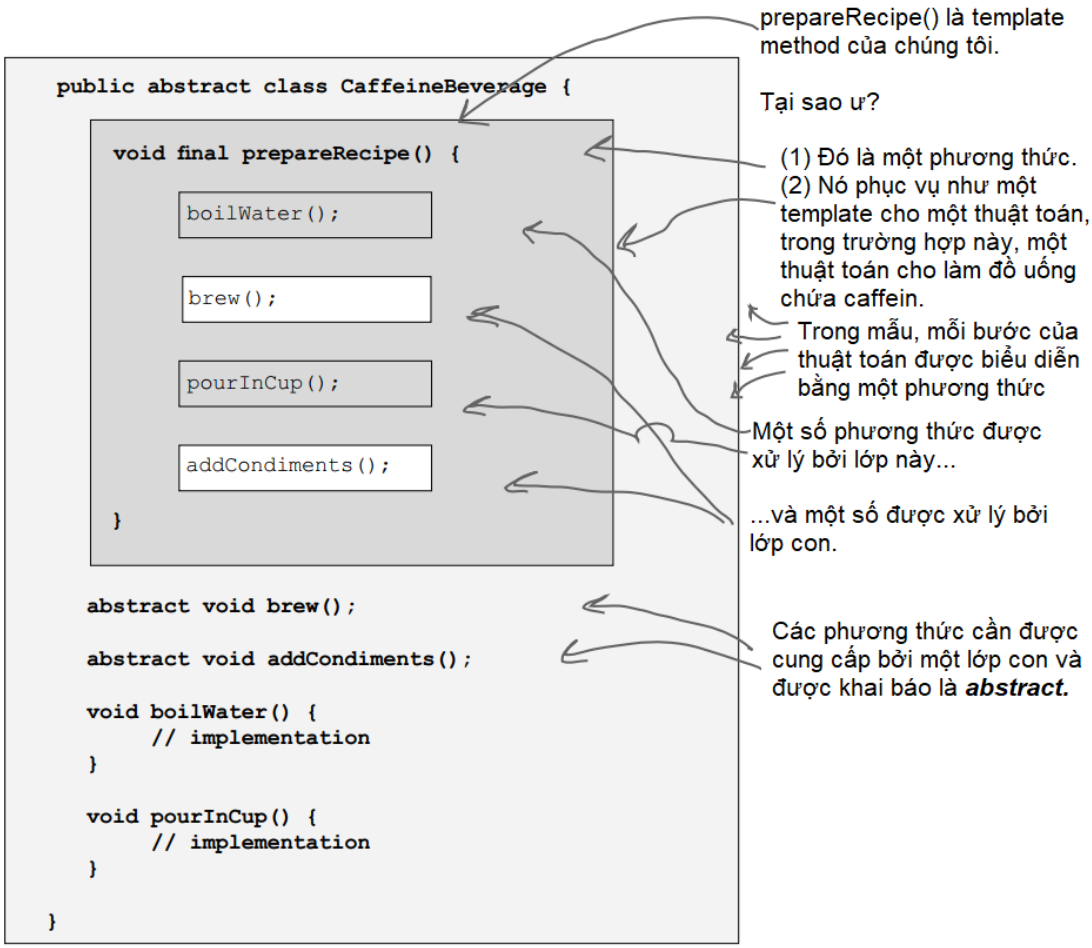
Bài tập: Bây giờ hãy vẽ sơ đồ lớp mới mà chúng tôi đã chuyển việc triển khai prepareRecipe() trong lớp CaffeineBeverage

Chúng ta đã làm gì?



Gặp gỡ Template Method Pattern

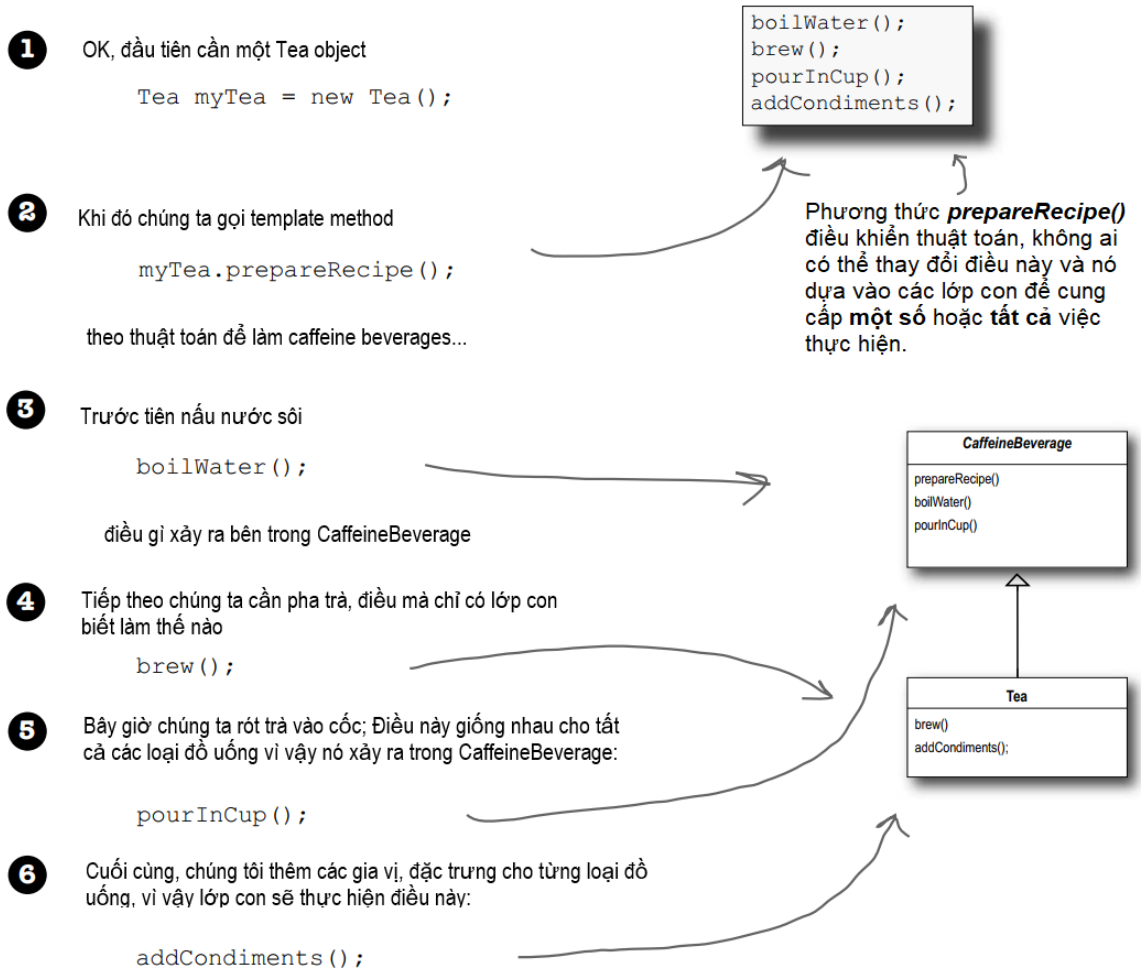
Về cơ bản, chúng tôi chỉ thực hiện **Template Method Pattern**. Đây là gì vậy? Hãy cùng xem xét cấu trúc của lớp **CaffeineBeverage**; nó chứa **Template Method Pattern** trên thực tế:



Template Method xác định các bước của thuật toán và cho phép các lớp con cung cấp việc thực hiện cho một hoặc nhiều bước đó.

Pha một ít trà bằng Template method pattern

Hãy tiếp tục bằng việc pha trà và theo dõi cách thức hoạt động của Template Method. Bạn có thể thấy rằng Template Method điều khiển thuật toán; tại một số điểm nhất định trong thuật toán, nó cho phép lớp con cung cấp việc thực hiện các bước…



Template Method Pattern đã cho chúng ta những gì?

Tea & Coffee implementation	CaffeineBeverage mới, được cung cấp bởi Template Method
Cà phê và trà đang tự ra quyết định; chúng kiểm soát thuật toán.	Lớp CaffeineBeverage tự ra quyết định; nó có thuật toán, và bảo vệ chúng.
Code bị trùng lặp ở lớp Coffee và Tea.	Lớp CaffeineBeverage tối đa hóa việc tái sử dụng giữa các lớp con.
Code thay đổi thuật toán yêu cầu mở tất cả các lớp con và thực hiện nhiều thay đổi.	Thuật toán chỉ ở một nơi (Template method) và thay đổi code chỉ cần được thực hiện thay đổi ở chính nơi đó.
Các lớp được tổ chức theo cấu trúc đòi hỏi nhiều công việc để thêm một loại thức uống caffeine mới.	Phiên bản Template Method cung cấp một framework mà các thức uống caffeine khác có thể được gắn vào. Caffeine beverage mới chỉ cần implement một vài phương thức.
Kiến thức về thuật toán và cách thực hiện nó được phân phối trên nhiều lớp.	Lớp CaffeineBeverage tập trung kiến thức về thuật toán và dựa vào các lớp con để cung cấp các triển khai hoàn chỉnh.

Định nghĩa Template Method Pattern

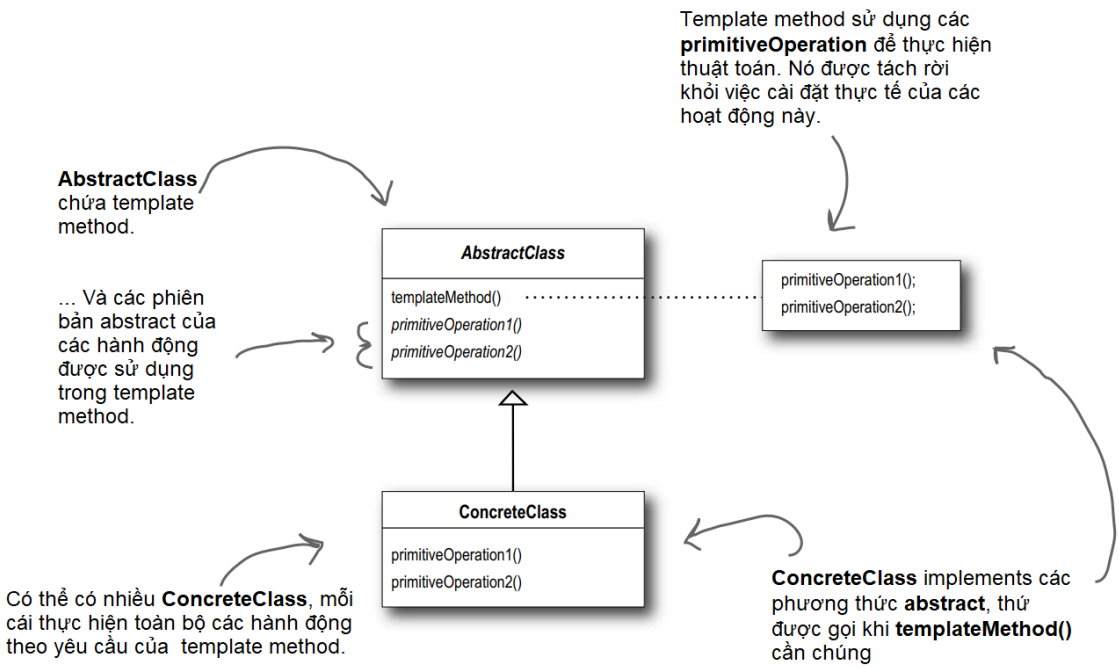
Bạn đã thấy cách thức **Template Method Pattern** hoạt động trong ví dụ về Trà và Cà phê của chúng tôi; Bây giờ, hãy xem định nghĩa chính thức và tìm hiểu chi tiết:

> **The Template Method Pattern** defines the skeleton of an algorithm in a method, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

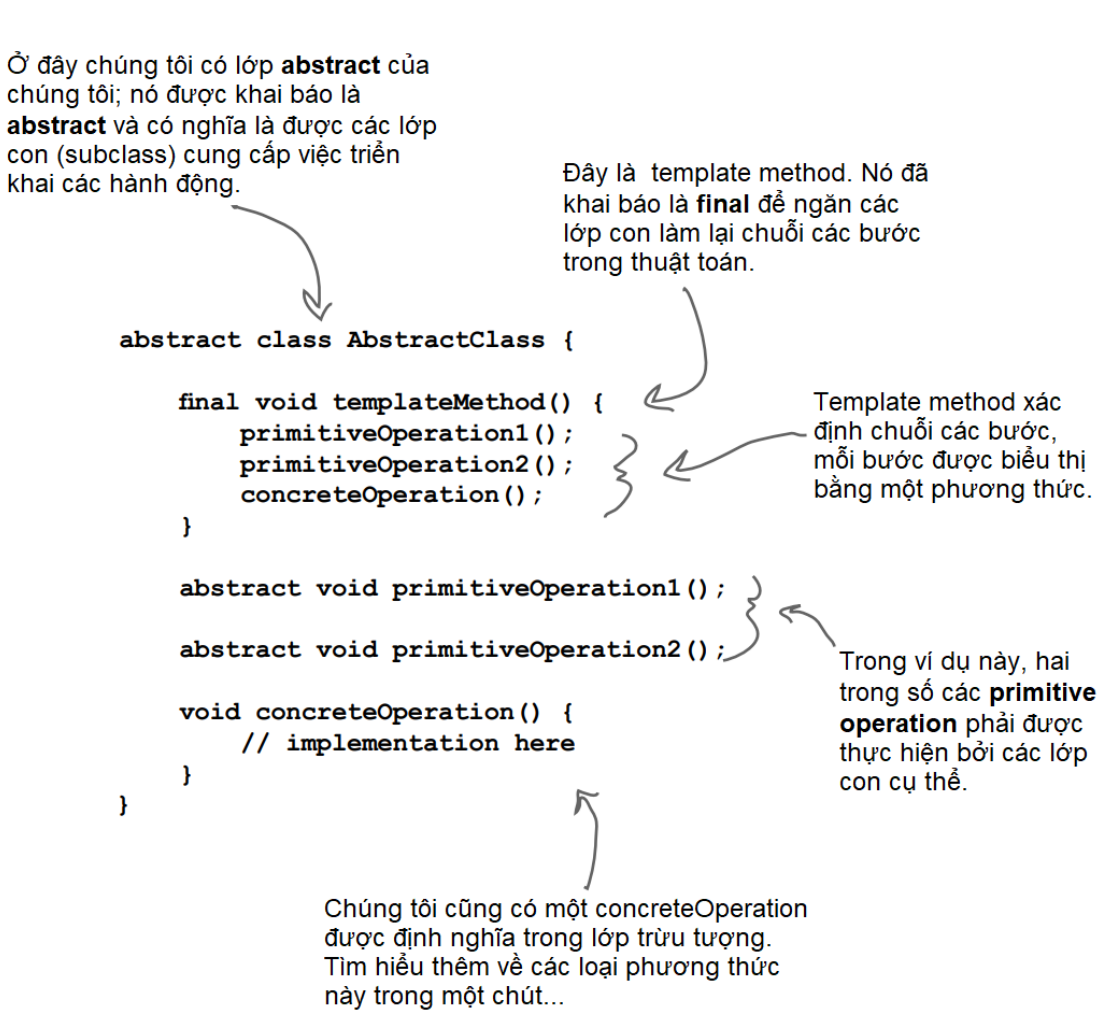
(Template Method Pattern xác định bộ khung của thuật toán trong một phương thức, trì hoãn một số bước cho các lớp con. Template Method cho phép các lớp con định nghĩa lại các bước nhất định của thuật toán mà không thay đổi cấu trúc thuật toán)

Mẫu này là tất cả về việc tạo một template cho một thuật toán. Template gì? Như bạn đã thấy nó chỉ là một phương thức; cụ thể hơn, nó là một phương thức định nghĩa thuật toán là một tập hợp các bước. Một hoặc nhiều bước trong số này được định nghĩa là abstract và được implement bởi một lớp con. Điều này đảm bảo cấu trúc thuật toán không thay đổi, trong khi các lớp con cung cấp một phần của việc thực hiện.

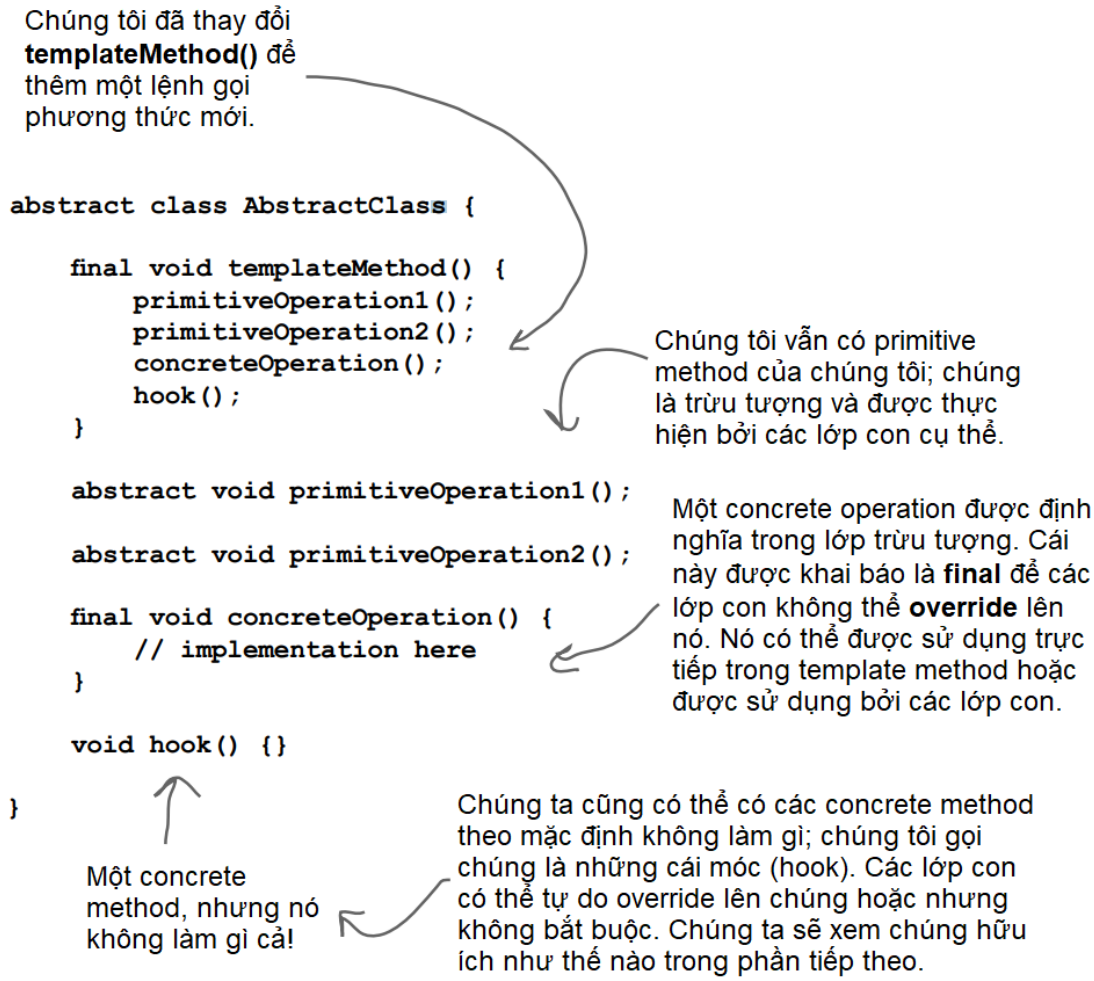
Template method pattern: Sơ đồ lớp



Chúng ta hãy xem xét kỹ hơn về cách định nghĩa của **AbstractClass**, bao gồm cả **templateMethod()** và các **primitiveOperation**.



Bây giờ chúng ta sẽ xem xét kỹ hơn các loại phương thức có thể đặt trong lớp abstract:



Hooked trên Template Method Pattern...

Với một hook, tôi có thể override phương thức, hoặc không. Nó là lựa chọn của tôi. Nếu tôi không muốn, lớp trừu tượng cung cấp một cài đặt mặc định.

Một hook là một phương thức được khai báo trong lớp trừu tượng, nhưng chỉ cung cấp một cài đặt mặc định hoặc rỗng. Điều này mang lại cho các lớp con khả năng móc nối vào thuật toán tại các điểm khác nhau, nếu chúng muốn; một lớp con cũng được tự do bỏ qua hook.

Có một số cách sử dụng hook; Hãy nhìn chúng ngay bây giờ. Chúng tôi sẽ nói về một vài cách sử dụng khác sau:

```
public abstract class CaffeineBeverageWithHook {

    final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        if (customerWantsCondiments()) {
            addCondiments();
        }
    }

    abstract void brew();

    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }

    void pourInCup() {
        System.out.println("Pouring into cup");
    }

    boolean customerWantsCondiments() {
        return true;
    }
}
```

Chúng tôi đã bổ sung một statement có điều kiện nhỏ dựa trên return của nó trên một phương thức cụ thể customerWantsCondiment(). Nếu client muốn add condiment, khi đó chúng ta chỉ cần gọi addCondiment()

Ở đây, chúng tôi đã định nghĩa một phương thức với một triển khai mặc định (hầu hết) trống. Phương thức này chỉ trả về true và không làm gì khác.

Đây là một **hook** bởi vì lớp con có thể ghi đè phương thức này, nhưng không cần thiết.

SỬ DỤNG HOOK

Để sử dụng hook, chúng ta override nó trong lớp con của chúng ta. Ở đây, hook kiểm soát xem **CaffeineBeverage** có phải là một phần nhất định của thuật toán hay không; đó là, liệu nó thêm một gia vị cho đồ uống không?

Làm thế nào để chúng ta biết liệu client muốn gia vị (đường, sữa, chanh…) không? Phải hỏi thôi!

```
public class CoffeeWithHook extends CaffeineBeverageWithHook {

    public void brew() {
        System.out.println("Dripping Coffee through filter");
    }

    public void addCondiments() {
        System.out.println("Adding Sugar and Milk");
    }

    public boolean customerWantsCondiments() {
        String answer = getUserInput();

        if (answer.toLowerCase().startsWith("y")) {
            return true;
        } else {
            return false;
        }
    }

    private String getUserInput() {
        String answer = null;

        System.out.print("Would you like milk and sugar with your coffee (y/n)? ");

        BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
        try {
            answer = in.readLine();
        } catch (IOException ioe) {
            System.err.println("IO error trying to read your answer");
        }
        if (answer == null) {
            return "no";
        }
        return answer;
    }
}
```

Tại đây, nơi bạn override lên hook và cung cấp chức năng của riêng bạn.

Nhận đầu vào của người dùng trên quyết định có thêm toping không và trả về **true** hay **false**, tùy thuộc vào đầu vào.

Đoạn này hỏi người dùng anh ấy thích sữa và đường trong cà phê không? và nhận đầu vào từ command line.

HÃY RUN TESTDRIVE

Được rồi, nước đang sôi … Ở đây, code thử nghiệm nơi chúng tôi tạo ra một tách trà và một tách cà phê nóng.

```
public class BeverageTestDrive {
    public static void main(String[] args) {

        TeaWithHook teaHook = new TeaWithHook();
        CoffeeWithHook coffeeHook = new CoffeeWithHook();

        System.out.println("\nMaking tea...");
        teaHook.prepareRecipe();

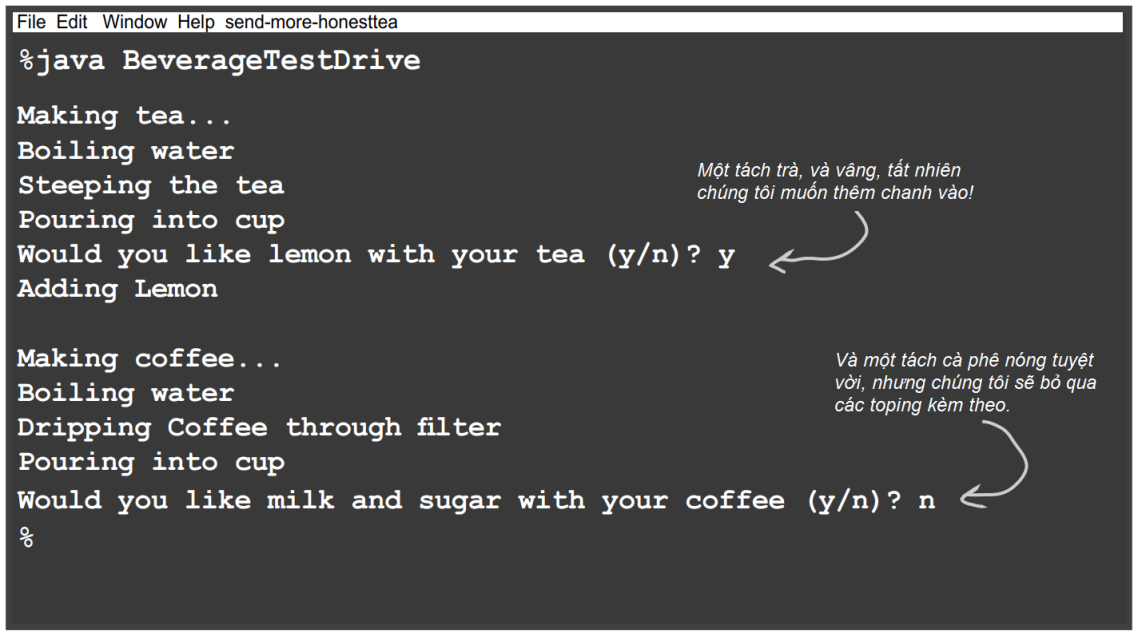
        System.out.println("\nMaking coffee...");
        coffeeHook.prepareRecipe();
    }
}
```

← Create a tea.

← A coffee.

And call prepareRecipe() on both!

Và hãy để cho nó chạy…



Bây giờ, tôi có thể nghĩ rằng chức năng như hỏi client có thể đã được sử dụng bởi tất cả các lớp con không?

Bạn biết gì không? Chúng tôi đồng ý với bạn. Nhưng bạn phải thừa nhận đó là một ví dụ khá hay về cách một hook có thể được sử dụng để kiểm soát một cách có điều kiện dòng chảy của thuật toán trong lớp abstract. Chúng tôi chắc chắn rằng bạn có thể nghĩ ra nhiều kịch bản thực tế khác, nơi bạn có thể sử dụng template method và hook trong code của riêng bạn.

Không có câu hỏi ngớ ngẩn

Hỏi: Khi tôi tạo một **template method**, làm thế nào để tôi biết khi nào nên sử dụng abstract method và khi nào sử dụng hook?

Trả lời: Sử dụng các abstract method khi lớp con của bạn **BẮT BUỘC** cung cấp việc triển khai phương thức hoặc các bước trong thuật toán. Sử dụng hook khi phần đó của thuật toán là tùy chọn (có thể thực hiện hoặc không thực hiện do lớp con lựa chọn). Với hook, một lớp con có thể chọn thực hiện hook đó, nhưng nó cũng có thể không cần thực hiện.

Hỏi: Hook thực sự được cho là dùng để làm gì?

Trả lời: Có một vài cách sử dụng hook. Như chúng ta vừa nói, một hook có thể cung cấp một cách để một lớp con thực hiện một phần tùy chọn của thuật toán hoặc nếu nó không quan trọng đối với việc thực hiện ở lớp con thì lớp con có thể bỏ qua nó. Một cách sử dụng khác là tạo cho lớp con cơ hội phản ứng với một số bước trong template method sắp xảy ra hoặc vừa xảy ra. Chẳng hạn, một phương thức hook như **justReOrderedList()** cho phép lớp con thực hiện một số hoạt động (chẳng hạn như hiển thị lại hiển thị trên màn hình) sau khi danh sách được reoder (sắp xếp lại). Như bạn đã thấy một cái hook cũng có thể cung cấp một lớp con với khả năng đưa ra quyết định cho lớp trừu tượng.

Hỏi: Một lớp con có phải implement tất cả các abstract method trong **AbstractClass** không?

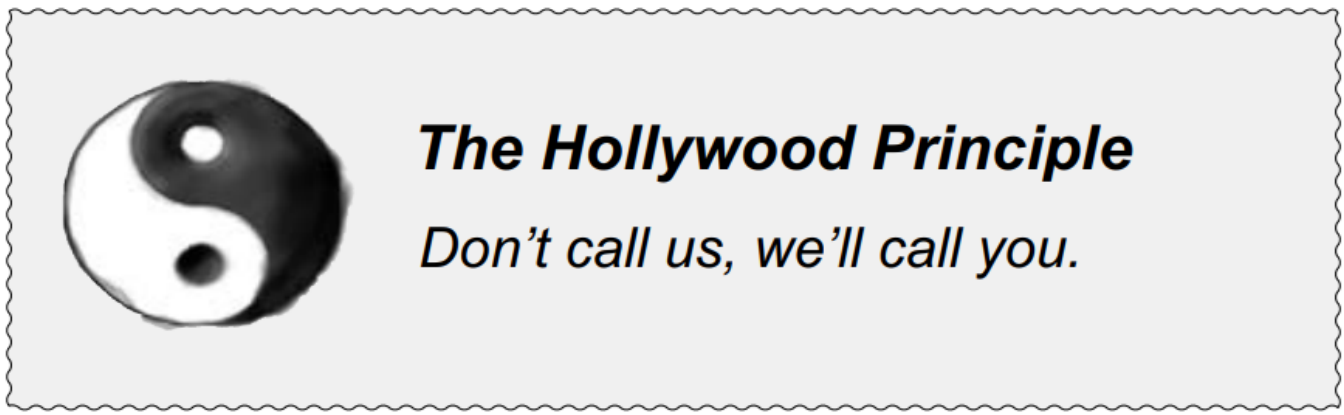
Trả lời: Có, mỗi lớp con cụ thể xác định toàn bộ tập hợp các abstract method và cung cấp một triển khai đầy đủ các bước chưa được định nghĩa của thuật toán template method.

Hỏi: Có vẻ như tôi nên giữ các abstract method của mình với số lượng nhỏ, nếu không, sẽ là một công việc lớn để implement chúng trong lớp con.

Trả lời: Đó là một điều tốt để ghi nhớ khi bạn viết các template method. Đôi khi điều này có thể được thực hiện bằng cách không làm cho các bước của thuật toán của bạn quá chi tiết. Nhưng nó rõ ràng là một sự đánh đổi: càng ít chi tiết, càng kém linh hoạt. Hãy nhớ rằng, một số bước sẽ là tùy chọn; vì vậy bạn có thể thực hiện chúng như các hook thay vì các phương thức trừu tượng, giảm bớt gánh nặng cho các lớp con của lớp trừu tượng của bạn.

Nguyên tắc từ Hollywood

Chúng tôi đã có một nguyên tắc thiết kế khác cho bạn; nó gọi là Nguyên tắc Hollywood:

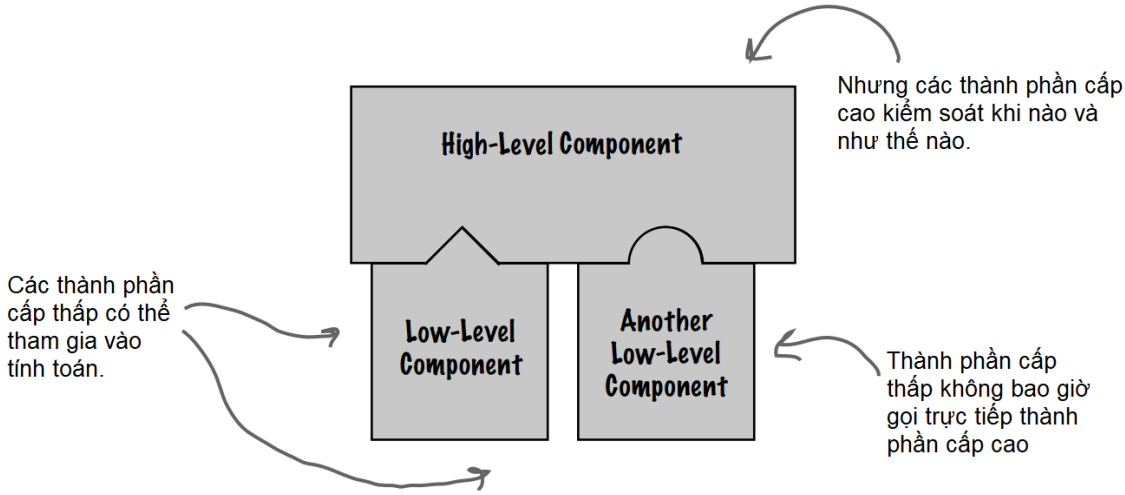


Đừng gọi chúng tôi, chúng tôi sẽ gọi bạn.

Dễ nhớ phải không? Nhưng nó có liên quan gì đến thiết kế OO?

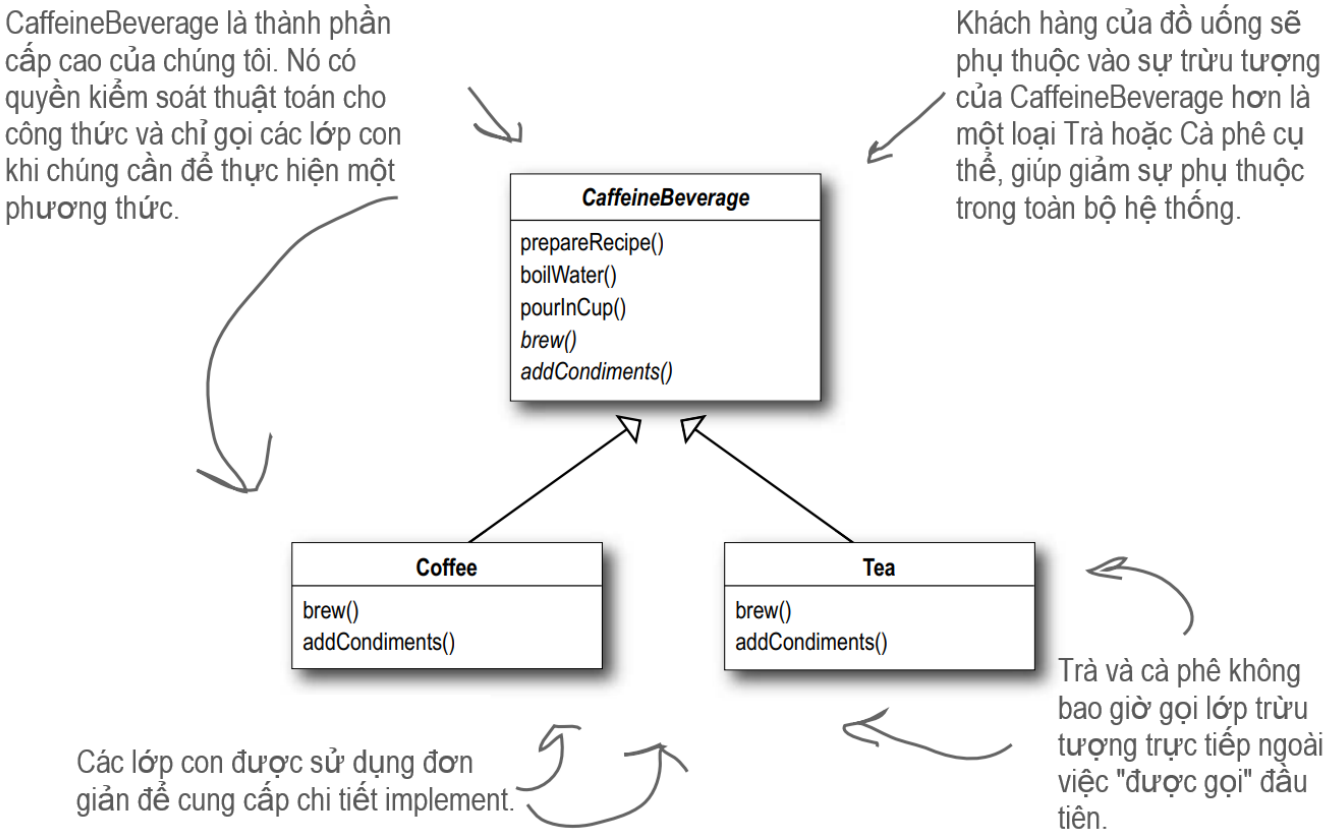
Nguyên tắc Hollywood cung cấp cho chúng ta một cách để ngăn chặn "dependency rot" (nghĩa đen: Sự thối rữa phụ thuộc – hiểu nôm na là sự phụ thuộc ảnh hưởng đến chúng ta, ví dụ khi bảo trì sẽ khó khăn hơn…). "Dependency rot" xảy ra khi bạn có các thành phần cấp cao phụ thuộc vào các thành phần cấp thấp, phụ thuộc tiếp vào các thành phần cấp cao khác phụ thuộc vào các thành cấp thấp, v.v. . Khi "Dependency rot" xảy ra, không ai có thể dễ dàng hiểu cách thiết kế một hệ thống.

Với Nguyên tắc Hollywood, chúng tôi cho phép các thành phần cấp thấp tự hook vào một hệ thống, nhưng các thành phần cấp cao xác định khi nào cần thiết và làm thế nào. Nói cách khác, các thành phần cấp cao cung cấp cho các thành phần cấp thấp một tên "đừng gọi chúng tôi, chúng tôi sẽ gọi bạn" (don't call us, we'll call you).



Hollywood Principle và Template Method Pattern

Mối liên hệ giữa Nguyên tắc Hollywood và Template method pattern có lẽ đã rõ ràng: khi chúng tôi thiết kế với Template method, chúng tôi sẽ nói với các lớp con, "*don't call us, we'll call you*". Bằng cách nào? Hãy xem xét lại thiết kế **CaffeineBeverage** của chúng tôi:



Sử dụng sức mạnh bộ não

Những mẫu thiết kế nào khác sử dụng Nguyên tắc Hollywood? Factory Method (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html), Observer (https://toihocdesignpattern.com/head-first-design-patterns-tieng-viet-chuong-2-observer-pattern.html); hay bất kì mẫu khác?

Không có câu hỏi ngớ ngẩn

Hỏi: Nguyên tắc Hollywood và Nguyên tắc nghịch đảo phụ thuộc (Dependency inversion principle) mà chúng ta đã học được một vài chương trước liên quan như thế nào?

Trả lời: Nguyên tắc nghịch đảo phụ thuộc dạy chúng ta tránh sử dụng các lớp cụ thể và thay vào đó làm việc nhiều nhất có thể với sự trừu tượng. Nguyên tắc Hollywood là một kỹ thuật xây dựng các framework hoặc các component để các thành phần cấp thấp hơn có thể được nối (hook) vào xử lý, nhưng không tạo ra sự phụ thuộc giữa các thành phần cấp thấp vào các lớp cấp cao hơn.

Vì vậy, cả hai đều có mục tiêu là tách rời, nhưng Nguyên tắc nghịch đảo phụ thuộc tạo nên một tuyên bố chung mạnh mẽ và mạnh mẽ hơn nhiều về Làm thế nào để tránh phụ thuộc trong thiết kế. Nguyên tắc Hollywood cho chúng ta một kỹ thuật để tạo ra các thiết kế cho phép các cấu trúc cấp thấp tương tác với nhau trong khi ngăn các lớp khác trở nên quá phụ thuộc vào chúng.

Hỏi: Có phải thành phần cấp thấp không được phép gọi phương thức trong thành phần cấp cao hơn không?

Trả lời: Không hẳn. Trong thực tế, một thành phần cấp thấp thường sẽ gọi một phương thức được định nghĩa ở trên nó trong hệ thống phân cấp kế thừa hoàn toàn thông qua kế thừa. Nhưng chúng tôi muốn tránh tạo ra sự phụ thuộc vòng tròn rõ ràng giữa thành phần cấp thấp và thành phần cấp cao.

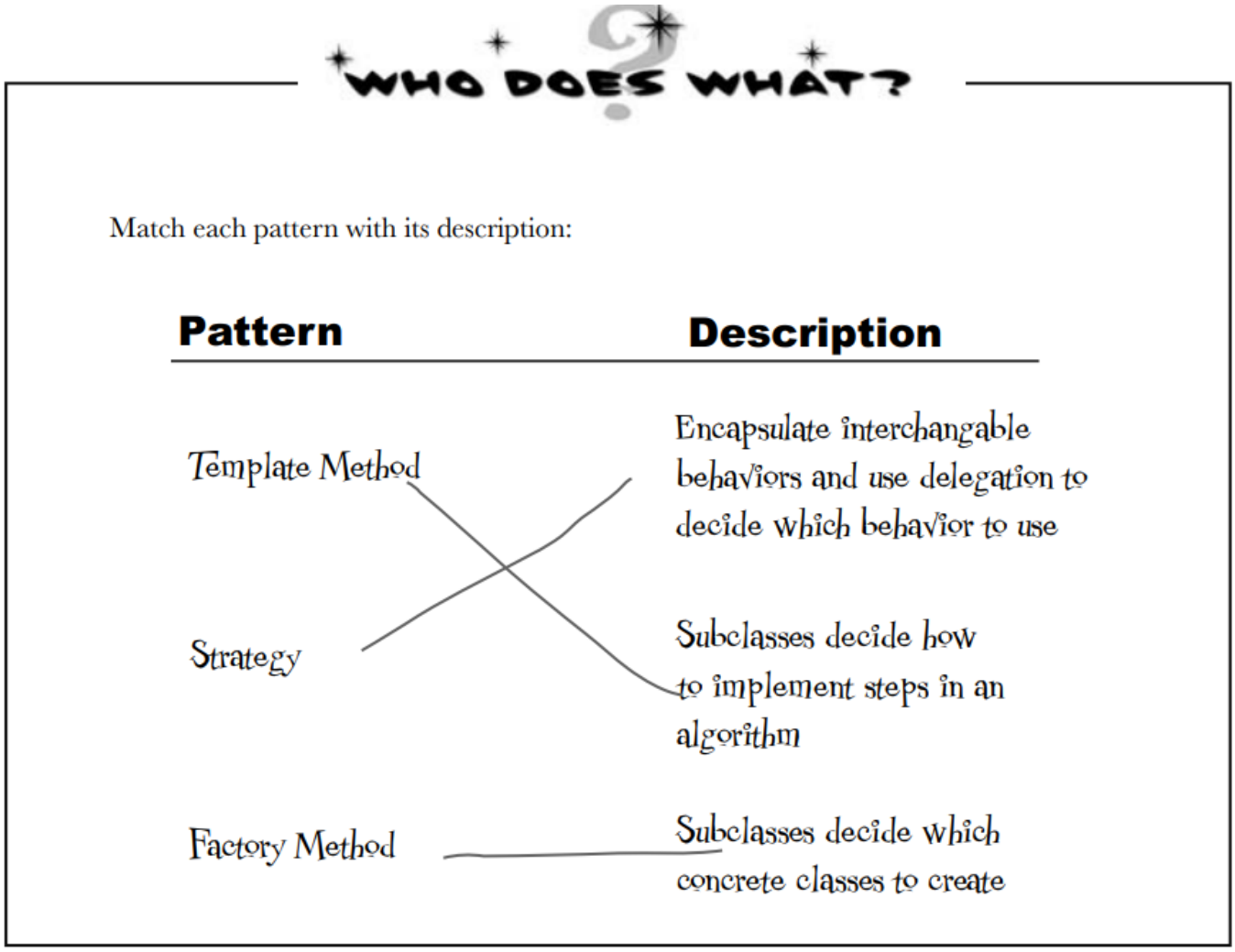
Bài tập:

WHO DOES WHAT?

Match each pattern with its description:

Pattern	Description
Template Method	Encapsulate interchangeable behaviors and use delegation to decide which behavior to use
Strategy	Subclasses decide how to implement steps in an algorithm
Factory Method	Subclasses decide which concrete classes to create

Đáp án:



Template Method Pattern trong thực tế

Template Method Pattern là một mẫu rất phổ biến và bạn sẽ tìm thấy rất nhiều mẫu trong thực tế. Mặc dù vậy, bạn phải có một con mắt sắc sảo, bởi vì có nhiều cách triển khai Template Method không hoàn toàn trông giống như thiết kế trong sách giáo khoa.

Mẫu này xuất hiện thường xuyên vì đây là công cụ thiết kế tuyệt vời để tạo framework, trong đó framework kiểm soát cách thực hiện, nhưng để bạn (người sử dụng framework) xác định chi tiết của riêng bạn về những gì thực sự xảy ra ở mỗi bước của thuật toán của framework.

Hãy xem Safari thông qua một vài cách sử dụng trong thực tế (tốt, được thôi, trong API Java)…



Sorting với Template Method Pattern

Cái gì mà chúng ta thường cần phải làm với mảng?

Sắp xếp chúng!

Nhận thấy điều đó, các nhà thiết kế của lớp Java Arrays đã cung cấp cho chúng ta một template tiện dụng để sắp xếp. Chúng ta hãy xem cách thức hoạt động của phương thức này:
(Chúng tôi đã phân tích code này một chút để dễ giải thích hơn. Nếu bạn muốn xem tất cả, hãy lấy nguồn từ Sun và xem thử…)

Chúng tôi thực sự có hai phương thức ở đây và chúng hoạt động cùng nhau để cung cấp chức năng sắp xếp

Phương thức đầu tiên, sort(), chỉ là một phương thức helper giúp tạo một copy của mảng và chuyển nó dọc theo như mảng đích dest[ ] cho phương thức mergeSort(). Nó cũng đi dọc theo chiều dài của mảng và cho biết sắp xếp bắt đầu từ phần tử đầu tiên.

```
public static void sort(Object[] a) {
    Object aux[] = (Object[])a.clone();
    mergeSort(aux, a, 0, a.length, 0);
}
```

Phương thức mergeSort() chứa thuật toán sắp xếp và dựa vào việc triển khai phương thức compareTo() để hoàn thành thuật toán. Nếu bạn có hứng thú với sự khó chịu của việc sắp xếp xảy ra, bạn hãy kiểm tra mã nguồn của Sun.

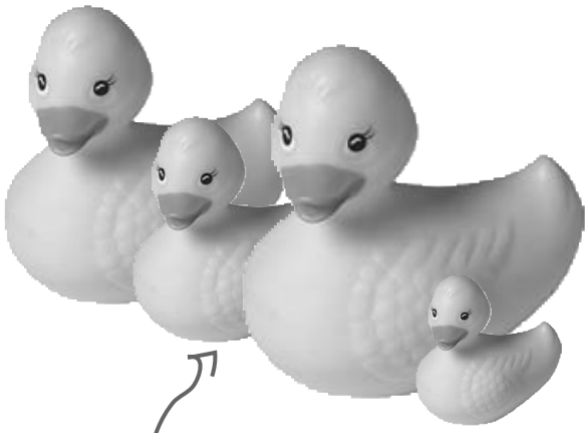
Think of this as the **template method**.

```
private static void mergeSort(Object src[], Object dest[],
             int low, int high, int off)
{

    for (int i=low; i<high; i++){
        for (int j=i; j>low &&
             ((Comparable)dest[j-1]).compareTo((Comparable)dest[j])>0; j--)
        {
            swap(dest, j, j-1);
        }
    }
    return;
}
```

This is a concrete method, already defined in the Arrays class.

compareTo() is the method we need to implement to "fill out" the template method.



We've got an array of Ducks we need to sort.

Chúng tôi đã có một số vịt để sắp xếp...

Hãy nói rằng bạn có một đàn vịt mà bạn muốn sắp xếp. Làm sao bạn làm điều đó? Chà, sort template method trong Arrays cung cấp cho chúng tôi thuật toán, nhưng bạn cần cho nó biết cách so sánh vịt, mà bạn làm bằng cách implementing phương thức **compareTo()** của lớp Arrays… Như vậy có hợp lý không?

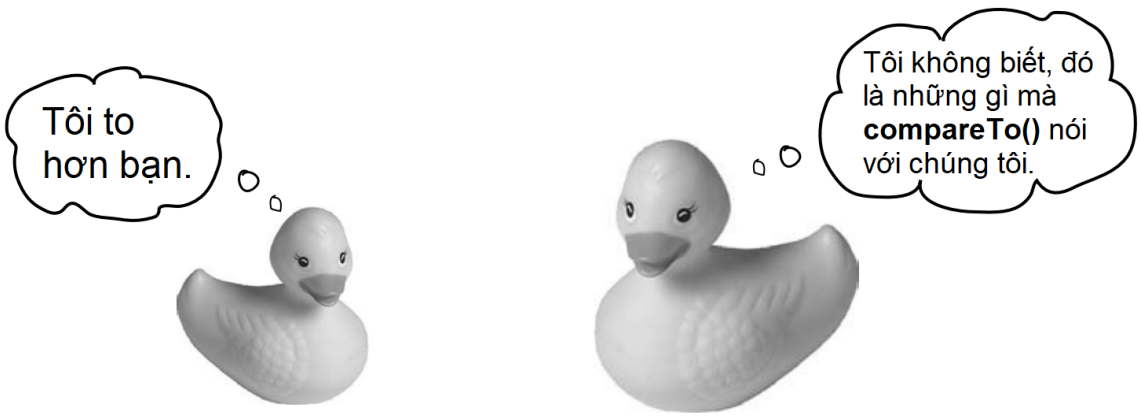
Không, nó không hợp lí. Một Array sẽ không có lớp con nào. Tôi nghĩ rằng **sort()** chính là **templateMethod()**. Và bên trong phương thức sort có gọi đến **compareTo()**, đây chính là một bước trong thuật toán và cần các lớp con override lại.

Ở đây, một thỏa thuận: các nhà thiết kế **sort()** muốn nó hữu ích trên tất cả các mảng, vì vậy họ phải tạo một phương thức **static sort()** có thể được sử dụng từ bất cứ đâu. Nhưng điều đó không sao, nó hoạt động gần giống như khi nó ở trong một superclass. Bây giờ, đây là một chi tiết nữa: bởi vì **sort()** thực sự không được định nghĩa trong superclass của chúng ta, nên phương thức **sort()** cần biết rằng bạn đã triển khai phương thức **compareTo()** hoặc nếu không bạn không có phần cần thiết để hoàn thành thuật toán sắp xếp.

Để xử lý điều này, các nhà thiết kế đã sử dụng **Comparable** interface. Tất cả những gì bạn phải làm là implement interface này, nó có một phương thức (bất ngờ chưa): **compareTo()**.

compareTo() là gì?

Phương thức **compareTo()** so sánh hai đối tượng và trả về khi một đối tượng nhỏ hơn, lớn hơn hoặc bằng đối tượng kia. **sort()** sử dụng điều này làm cơ sở so sánh các đối tượng trong mảng.



So sánh Duck với Duck

Được rồi, vì vậy bạn biết rằng nếu bạn muốn sắp xếp Vịt, bạn sẽ phải implements phương thức **compareTo()** này; bằng cách đó, bạn sẽ cung cấp cho lớp Arrays những gì nó cần để hoàn thành thuật toán và sắp xếp vịt của bạn.

Đây là cách thực hiện:

Hãy nhớ rằng, chúng ta cần triển khai giao diện **Comparable** vì chúng ta không thực sự phân lớp

```
public class Duck implements Comparable {
    String name;
    int weight;

    public Duck(String name, int weight) {
        this.name = name;
        this.weight = weight;
    }

    public String toString() {
        return name + " weighs " + weight;
    }
```

Vịt của chúng tôi có tên (name) và trọng lượng (weight)

Chúng tôi tiếp tục giữ nó đơn giản; tất cả những con vịt làm là in tên và trọng lượng của chúng!

Được rồi, ở đây, những gì phương thức sort cần...

```
    public int compareTo(Object object) {

        Duck otherDuck = (Duck)object;

        if (this.weight < otherDuck.weight) {
            return -1;
        } else if (this.weight == otherDuck.weight) {
            return 0;
        } else { // this.weight > otherDuck.weight
            return 1;
        }
    }
}
```

compareTo() lấy một con Vịt khác để so sánh với **this Duck**.

Ở đây, nơi chúng tôi chỉ định cách so sánh vịt. Nếu **this Duck** nhẹ hơn **otherDuck** thì chúng tôi trả về -1; nếu chúng bằng nhau, chúng ta trả về 0; và nếu **this Duck** nặng hơn, chúng tôi return 1.

Hãy sắp xếp lại những con Vịt

Đây là test drive cho việc sắp xếp Ducks…

```
public class DuckSortTestDrive {
    public static void main(String[] args) {
        Duck[] ducks = {
                        new Duck("Daffy", 8),
                        new Duck("Dewey", 2),
                        new Duck("Howard", 7),
                        new Duck("Louie", 2),
                        new Duck("Donald", 10),
                        new Duck("Huey", 2)
         };

        System.out.println("Before sorting:");
        display(ducks);

        Arrays.sort(ducks);

        System.out.println("\nAfter sorting:");
        display(ducks);
    }

    public static void display(Duck[] ducks) {
        for (int i = 0; i < ducks.length; i++) {
            System.out.println(ducks[i]);
        }
    }
}
```

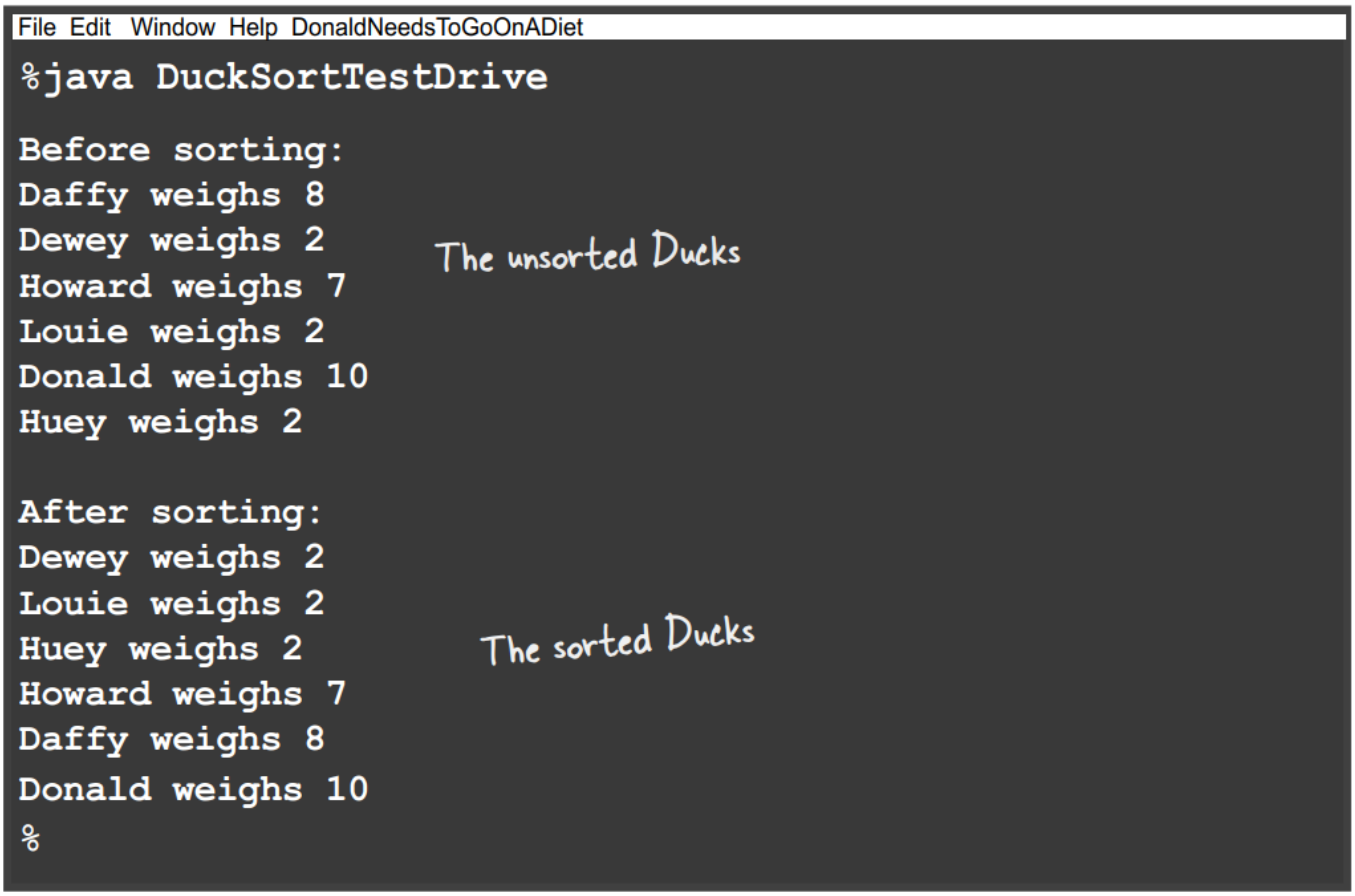
Chúng tôi cần một mảng những con vịt

Hãy in chúng để xem tên và trọng lượng của chúng.

Sắp xếp lại chúng!

Hãy in chúng (lần nữa) để xem lại tên và trọng lượng.

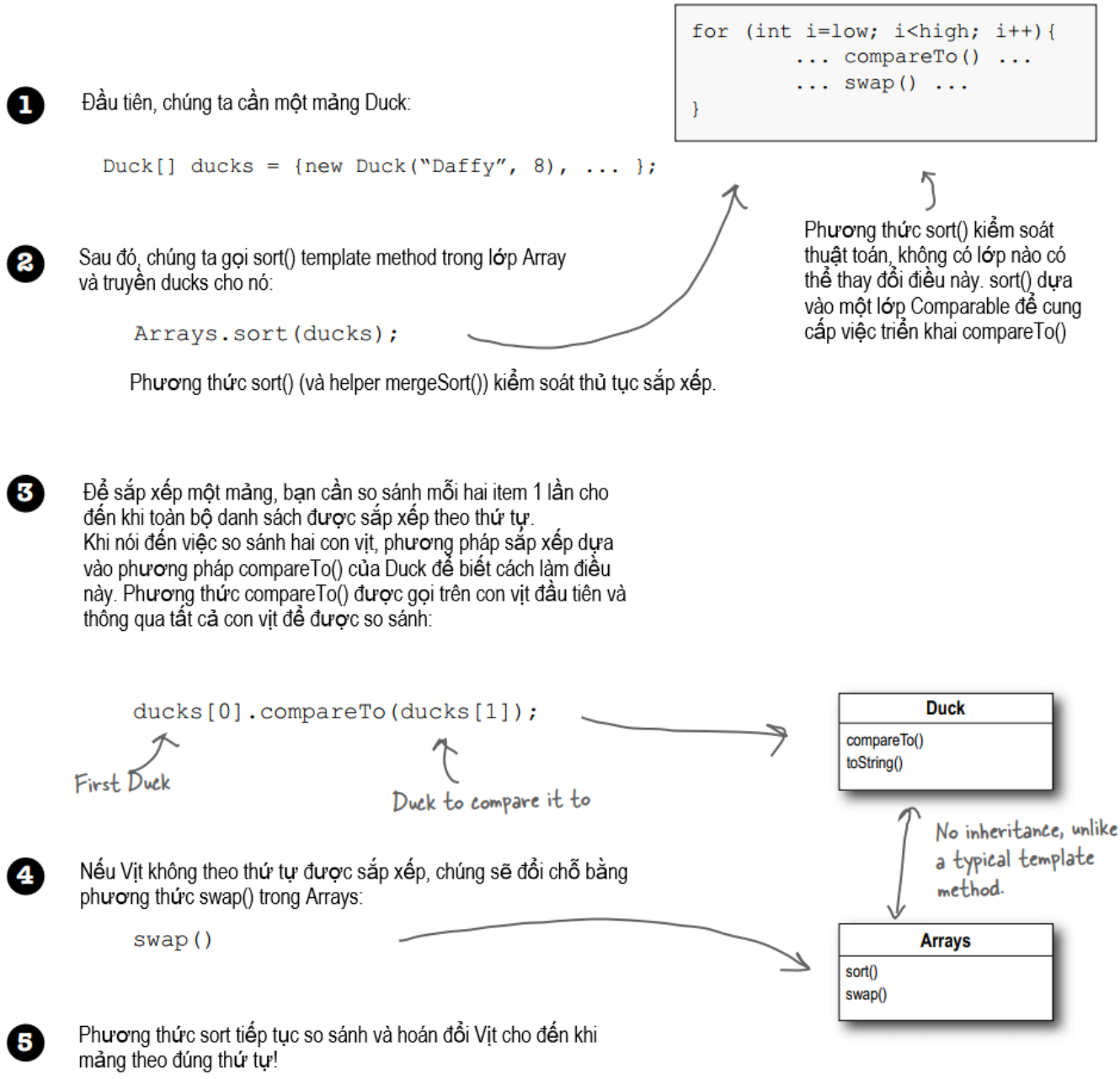
Lưu ý rằng chúng ta gọi static method **sort** của lớp Arrays, và truyền cho nó mảng Vịt của chúng ta.



Việc chế tạo máy phân loại vịt

Hãy để việc sắp xếp bắt đầu!

Thông qua cách thức hoạt động của **sort()** template method. Chúng tôi sẽ kiểm tra xem template method điều khiển thuật toán như thế nào và tại một số điểm nhất định trong thuật toán, xem cách nó yêu cầu Duck object của chúng tôi cung cấp việc thực hiện một bước tính toán…



Không có câu hỏi ngớ ngẩn

Hỏi: Đây có thực sự là **Template Method Pattern** hay bạn đang gượng ép quá mức?

Trả lời: Mẫu yêu cầu thực hiện một thuật toán và để các lớp con hoàn thành các bước trong thuật toán – và Arrays sort rõ ràng không làm điều đó! Nhưng, như chúng ta đã biết, các mẫu trong thực tế không phải lúc nào cũng giống như các mẫu trong sách giáo khoa. Chúng phải được sửa đổi để phù hợp với bối cảnh và các ràng buộc thực hiện.

Các nhà thiết kế của phương thức Arrays **sort()** có một vài ràng buộc. Nói chung, bạn không thể phân lớp một Java array và họ muốn phương thức sort được sử dụng trên tất cả các mảng (và mỗi mảng là một lớp khác nhau). Vì vậy, họ đã định nghĩa một phương thức static và chuyển phần so sánh của thuật toán tới các item được sắp xếp.

Vì vậy, mặc dù nó không phải là một template method pattern trong sách giáo khoa, nhưng việc triển khai này vẫn theo tinh thần của **Template Method Pattern**. Ngoài ra, bằng cách loại bỏ yêu cầu bạn phải phân lớp Array để sử dụng thuật toán này, họ đã thực hiện sắp xếp theo một số cách linh hoạt và hữu ích hơn.

Hỏi: Việc triển khai phương thức sort này thực sự có vẻ giống Strategy (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) hơn là **Template Method Pattern**. Tại sao chúng ta coi nó là **Template Method Pattern**?

Trả lời: Có lẽ bạn đang nghĩ rằng bởi vì Strategy (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html)Pattern sử dụng việc kết hợp (compose) đối tượng (Arrays compose với Comparable). Bạn nói đúng – chúng tôi đang sử dụng đối tượng Arrays để sắp xếp mảng của chúng tôi, do đó tương tự như Strategy. Nhưng hãy nhớ, trong Strategy, lớp mà bạn compose (lớp Comparable) sẽ thực hiện **toàn bộ thuật toán**. Trong khi thuật toán mà Arrays thực hiện để sắp xếp là **không toàn bộ**; nó cần một lớp con để cài đặt phương thức **compareTo()** còn thiếu. Vì vậy, nó giống như **Template Method Pattern** hơn.

Hỏi: Có ví dụ nào khác về các **Template Method Pattern** trong API Java không?

Trả lời: Vâng, bạn sẽ tìm thấy chúng ở một vài nơi. Ví dụ, **java.io** có một phương thức **read()** trong **InputStream** mà các lớp con phải thực hiện và được sử dụng bởi template method ***read(byte b[], int off, int len)***.

Sử dụng sức mạnh bộ não

Chúng ta biết rằng chúng ta nên ủng hộ compose hơn thừa kế, phải không? Chà, những người triển khai **sort()** template method đã quyết định không sử dụng tính kế thừa, và thay vào đó cài đặt **sort()** là một static method được compose với một **Comparable** khi chạy. Chúng tốt hơn như thế nào? Chúng tồi tệ hơn như thế nào? Làm thế nào để bạn biết nên tiếp cận cái này hay cái kia? Các mảng Java có làm cho điều này đặc biệt khó khăn không?

Sử dụng sức mạnh bộ não 2

Hãy nghĩ về một mẫu khác là cụ thể hóa của template method. Trong cụ thể hóa này, các **primitiveOperation** được sử dụng để tạo và return các đối tượng. Đây là mẫu gì? (Xem lại sơ đồ lớp để biết **primitiveOperation** là gì)

Swingin’ with Frames

Tiếp theo trên Template Method của chúng tôi… hãy chú ý đến việc xoay JFrames!

Nếu bạn không biết về JFrame, thì đó là Swing container cơ bản nhất và kế thừa phương thức **paint()**. Theo mặc định, **paint()** không làm gì cả vì nó là hook method! Bằng cách override **paint()**, bạn có thể chèn vào thuật toán JFrame, để hiển thị màn hình và có đầu ra đồ họa của riêng bạn được tích hợp vào JFrame. Ở đây, một ví dụ đơn giản về việc sử dụng JFrame để override phương thức hook **paint()**:

Chúng tôi mở rộng JFrame, trong đó có phương thức **update()** điều khiển thuật toán để cập nhật màn hình. Chúng ta có thể móc (hook) vào thuật toán đó bằng cách override phương thức **paint()**.

Đừng nhìn chi tiết! Chỉ cần một số khởi tạo ở đây...

Thuật toán cập nhật JFrame gọi **paint()**. Theo mặc định, **paint()** không làm gì cả ... nó là một cái móc. Chúng tôi đã override lại **paint()** và bảo JFrame vẽ một thông báo trong cửa sổ.

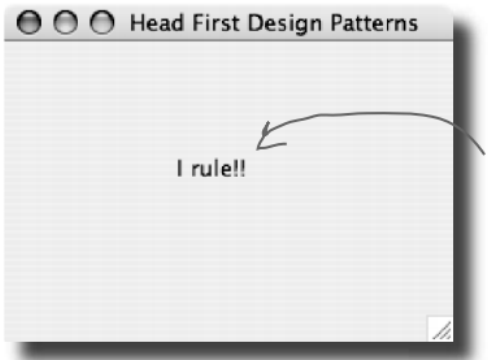
```
public class MyFrame extends JFrame {

    public MyFrame(String title) {
        super(title);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setSize(300,300);
        this.setVisible(true);
    }

    public void paint(Graphics graphics) {
        super.paint(graphics);
        String msg = "I rule!!";
        graphics.drawString(msg, 100, 100);
    }

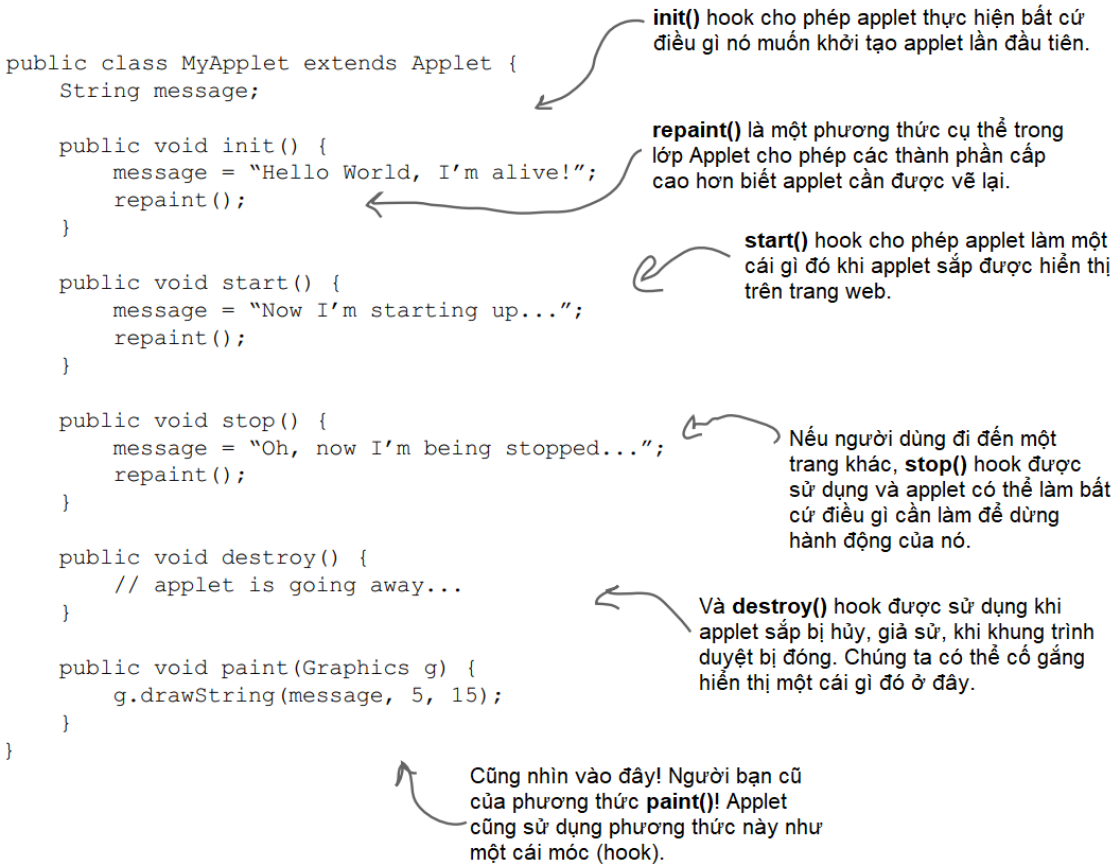
    public static void main(String[] args) {
        MyFrame myFrame = new MyFrame("Head First Design Patterns");
    }
}
```



Ở đây, thông điệp được vẽ lên cửa sổ vì chúng ta đã hook vào phương thức **paint()**.

Applets

Điểm dừng cuối cùng của chúng tôi: applet. Bạn có thể biết một applet là một chương trình nhỏ chạy trong một trang web. Bất kỳ applet nào cũng phải kế thừa **Applet** class và lớp này cung cấp một số hook. Hãy xem qua một vài trong số chúng:



Concrete Applet (lớp **MyApplet**) sử dụng các hook để cung cấp hành vi của riêng chúng. Bởi vì các phương thức này được cài đặt như các hook, applet không bắt buộc phải implement chúng.

Buổi nói chuyện tối nay: Template Method Pattern và Strategy Pattern (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) so sánh các method của họ

Template Method: Strategy (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html) này, chú em đang làm gì trong chương của tôi? Tôi hình dung tôi đã bị kẹt lại với một người nhàm chán như Factory Method (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html).

Factory Method: Này này tôi nghe được đấy, đừng có mà nói lén sau lưng tôi.

Strategy: Không, nó khác với tôi, mặc dù hãy cẩn thận – bạn và Factory Method (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html) có liên quan với nhau không?

Template Method: Tôi chỉ đùa thôi! Giờ thì nghiêm túc, chú em đang làm gì ở đây? Chúng tôi đã không nghe nói về chú em suốt tám chương rồi!

Strategy: Tôi nghe nói đến anh trong bản thảo của chương này và tôi nghĩ rằng tôi sẽ đi qua để xem nó diễn ra như thế nào. Chúng ta có rất nhiều điểm chung, vì vậy tôi nghĩ tôi có thể giúp…

Template Method: Chú em có thể muốn nhắc nhở người đọc một chút về những gì chú em làm, vì đã quá lâu để mọi người có thể nhớ chú em là ai.

Strategy: Tôi không biết, kể từ Chương 1 (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html), mọi người đã ngăn tôi trên phố và nói rằng, "Bạn không phải là mẫu đó phải không?…" vì vậy tôi nghĩ họ biết tôi là ai. Nhưng vì lợi ích của anh: Tôi định nghĩa một bộ các thuật toán và làm cho chúng có thể hoán đổi cho nhau. Vì mỗi thuật toán được đóng gói, client có thể sử dụng các thuật toán khác nhau một cách dễ dàng.

Template Method: Này, nghe có vẻ giống như những gì tôi làm. Nhưng ý định của tôi khác một chút so với chú em; công việc của tôi là xác định ra "bộ khung" của một thuật toán, nhưng để các lớp con của tôi thực hiện một số công việc còn lại. Bằng cách đó, tôi có thể có các triển khai khác nhau của các bước trong thuật toán, nhưng giữ quyền kiểm soát cấu trúc thuật toán. Có vẻ như chú em phải từ bỏ quyền kiểm soát các thuật toán của mình.

Strategy: Tôi không chắc chắn, nhưng dù sao, tôi cũng không bị phụ thuộc khi sử dụng tính kế thừa để triển khai thuật toán. Tôi cung cấp cho client một sự lựa chọn thực hiện thuật toán thông qua kết hợp (compose) đối tượng.

Template Method: Tôi nhớ điều đó, nhưng tôi có nhiều quyền kiểm soát hơn đối với thuật toán của mình và tôi không bị trùng lặp code. Trong thực tế, trong trường hợp mọi phần trong thuật toán đều giống nhau ngoại trừ một dòng nào đó khác nhau, thì các lớp của tôi hiệu quả hơn nhiều so với của chú em. Tất cả các code trùng lặp của tôi được đưa vào superclass, vì vậy tất cả các lớp con có thể chia sẻ nó.

Strategy: Anh có thể hiệu quả hơn một chút (chỉ một chút) và yêu cầu ít đối tượng hơn. Và anh cũng có thể ít phức tạp hơn một chút so với mô hình của tôi, nhưng tôi linh hoạt hơn vì tôi sử dụng object composition (kết hợp đối tượng). Với tôi, client có thể thay đổi thuật toán của họ khi chạy trong runtime chỉ bằng cách sử dụng một strategy object khác.

Template Method: Vâng, tôi rất vui cho chú em, nhưng đừng quên tôi là mẫu được sử dụng nhiều nhất. Tại sao? Bởi vì tôi cung cấp một phương thức cơ bản để tái sử dụng code cho phép các lớp con chỉ định hành vi. Tôi chắc chắn chú em có thể thấy rằng điều này là hoàn hảo để tạo các framework.

Strategy: Vâng, tôi đoán thế… nhưng, còn sự phụ thuộc thì sao? Anh phụ thuộc nhiều hơn tôi.

Template Method: Phụ thuộc như thế nào? Superclass của tôi là trừu tượng.

Strategy: Nhưng anh phải phụ thuộc vào các phương thức được triển khai trong superclass của anh, đây là một phần của thuật toán của anh. Tôi không phụ thuộc vào bất cứ ai; Tôi có thể tự làm toàn bộ thuật toán!

Template Method: Giống như tôi đã nói, Strategy (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html), tôi thật sự hạnh phúc cho chú em. Cảm ơn chú em đã ghé qua, nhưng tôi đã hoàn thành phần còn lại của chương này.

Strategy: Được rồi, được rồi, đừng quá xúc động. Tôi sẽ để cho anh làm việc, nhưng hãy cho tôi biết nếu anh cần các kỹ thuật đặc biệt của tôi, tôi sẽ luôn sẵn lòng giúp đỡ.

Template Method: Hiểu rồi. "Đừng gọi cho tôi, tôi sẽ gọi cho bạn…"

Tóm tắt

- Một "Template method" xác định các bước của một thuật toán, chuyển đến các lớp con để thực hiện các bước đó.
- Template method pattern cung cấp cho chúng ta một kỹ thuật quan trọng để tái sử dụng lại code.
- Lớp trừu tượng của Template method có thể định nghĩa các phương thức cụ thể, các phương thức trừu tượng và các hook.
- Các phương thức trừu tượng được thực hiện trong các lớp con.
- Hook là các phương thức không làm gì hoặc chứa hành vi mặc định trong lớp trừu tượng, nhưng có thể được override trong lớp con.
- Để ngăn các lớp con thay đổi thuật toán trong Template method pattern, hãy khai báo template method với từ khóa final.
- Nguyên tắc Hollywood hướng dẫn chúng ta đưa ra quyết định trong các mô-đun cấp cao có thể quyết định cách thức và thời điểm gọi các mô-đun cấp thấp.
- Bạn có thể thấy rất nhiều cách sử dụng Template Method Pattern trong thế giới thực, nhưng không mong đợi rằng tất cả (giống như bất kỳ mẫu nào) sẽ giống thiết kế bởi cuốn sách.
- Cả Strategy (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html)và Template Method Pattern đều đóng gói các thuật toán, một theo kế thừa và một theo compose.
- Factory Method (https://toihocdesignpattern.com/chuong-4-factory-pattern-va-abstract-factory-pattern-phan-1.html) là một cụ thể hóa của Template Method.